Proyecto Final

Ingeniería del software III
Grupo G1

Estudiantes:

2211881 Edgar Santiago Ariza Garcia
2211860 Javier Alejandro Silva Murillo
2211590 Oscar Fabian Fuentes Zuñiga

Presentado a:

Darío Alejandro Riaño Velandia

Bucaramanga
2026

1. Introducción

Este informe presenta el análisis técnico y funcional de la plataforma académica distribuida entregada como proyecto base de Ingeniería del software III. El sistema se compone de cinco microservicios Spring Boot que gestionan identidad, estudiantes, cursos, matrículas y calificaciones. El trabajo consistió en estudiar el código fuente, ejecutar el sistema con Docker Compose, verificar su comportamiento con peticiones HTTP y con las pruebas automáticas, y sustentar las decisiones de diseño.

Metodología aplicada:

- Lectura del código de cada servicio, siguiendo las capas controlador, handler, servicio y repositorio.
- Ejecución del stack completo con Docker Compose y peticiones a través de Swagger UI con los usuarios de prueba.
- Ejecución de las 27 pruebas automáticas (mvn test) y contraste de los resultados con el código.

2. Identificación de microservicios, módulos y tecnologías

El sistema está formado por cinco procesos independientes, sin módulo Maven agregador. Cada uno tiene su puerto, su base H2 en memoria y su contrato HTTP.

Servicio	Puerto	Base H2	Responsabilidad
auth-service	8081	authdb	Login con BCrypt, emisión del JWT HS256 y validación en GET /auth/validate
estudiantes-service	8082	estudiantesdb	CRUD de estudiantes, correo único y borrado condicionado
cursos-service	8083	cursosdb	CRUD de cursos, cupo máximo y borrado condicionado
matriculas-service	8084	matriculasdb	Alta y anulación de matrículas; verifica estudiante, curso y cupo
calificaciones-service	8085	calificacionesdb	Notas por corte y promedio ponderado

Tecnologías: Java 21, Spring Boot 3.2.5, Maven 3.9.9 (en contenedor), Spring Security con un filtro propio, JWT con firma HS256, BCrypt, OpenFeign, Spring Data JPA, base H2 en memoria, Swagger UI, Docker y Docker Compose.

Módulos internos de cada servicio de negocio: controller, handler, service, repository, entidades, dto, client (interfaces Feign), security y manejo de excepciones. Auth-service no tiene handler: su controlador llama directamente a AuthServiceImpl.

3. Arquitectura del sistema

Los cuatro servicios de negocio siguen la misma organización por capas: Controller, Handler, Service y Repository. El controlador no recibe ni devuelve entidades: el handler convierte el DTO de entrada en entidad y la entidad en DTO de salida. La sesión es stateless y la seguridad se aplica con un filtro antepuesto a la cadena de Spring Security.

Patrones y decisiones arquitectónicas identificados:

- Microservicios con una base de datos por servicio y referencias lógicas (Long) en lugar de claves foráneas entre procesos.
- Comunicación síncrona con OpenFeign y propagación del Bearer mediante FeignAuthPropagationConfig.
- DTO de entrada y de salida, y un sobre común de respuesta (ApiResponse) construido por ResponseBuilder.
- Manejo centralizado de errores con GlobalExceptionHandler, CustomAuthenticationEntryPoint y CustomAccessDeniedHandler.

Existe un ciclo de llamadas entre matrículas y cursos, y entre matrículas y estudiantes. Docker Compose no admite ciclos en depends_on, por eso estudiantes y cursos arrancan sin declarar a matrículas. El ciclo de llamadas existe, el ciclo de arranque no.

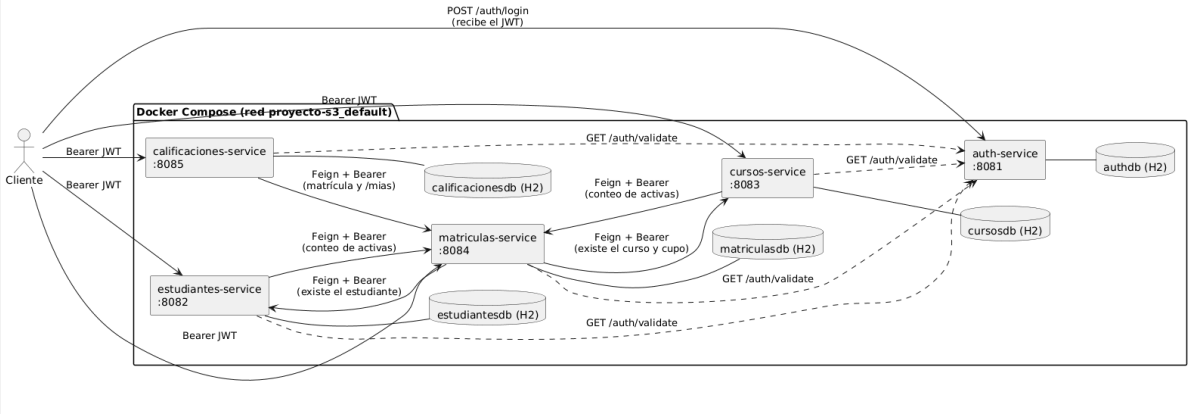


Diagrama de componentes: los 5 servicios, sus bases H2, las llamadas entre ellos y el recorrido del token

4. Seguridad y autenticación

Recorrido del token. El cliente inicia sesión en auth-service (POST /auth/login) y recibe un JWT HS256 con expiración de una hora. Los claims son sub, role, iat, exp y, solo si el usuario tiene estudiante asociado, estudianteId. En cada petición a un servicio de negocio, AuthValidationFilter llama a GET /auth/validate con el mismo Bearer y carga una autoridad igual al rol. El secreto de firma permanece únicamente en auth-service.

Credenciales. AuthServiceImpl.login lanza la misma InvalidCredentialsException tanto si el usuario no existe como si la contraseña es incorrecta, de modo que el cliente no puede distinguir ambos casos. Las contraseñas se guardan con BCryptPasswordEncoder, que incluye versión, costo y salt en el hash almacenado.

401 y 403. El 401 corresponde a autenticación fallida: header ausente (AUTH_HEADER_MISSING), token rechazado por auth-service (TOKEN_INVALID) o auth-service inalcanzable (TOKEN_VALIDATION_ERROR). El 403 corresponde a un usuario autenticado sin permiso, y lo responde CustomAccessDeniedHandler con AUTH_FORBIDDEN.

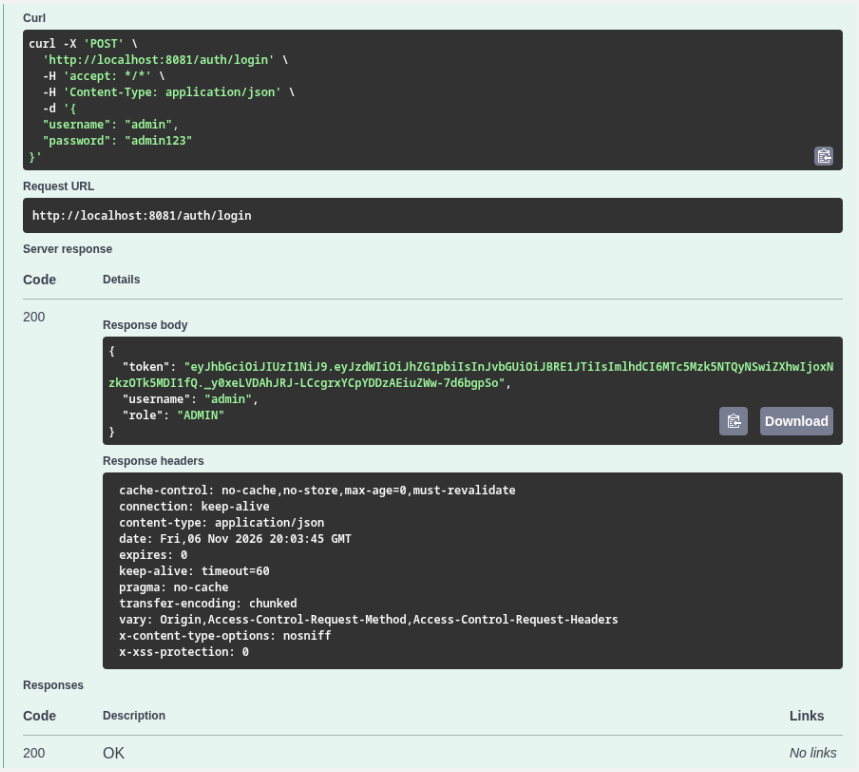
Riesgo identificado. El secreto HMAC está escrito como literal en el application.yaml de auth-service. Quien lo posea puede firmar tokens de cualquier rol. Se recomienda inyectarlo por variable de entorno y rotarlo; esta propuesta se desarrolla en la sección 14.

Roles y permisos:

Operación	ADMIN	DOCENTE	ESTUDIANTE
Leer estudiantes	Sí	Sí	No
Leer cursos	Sí	Sí	Sí
Escribir estudiantes y cursos	Sí	No	No
Matrículas: lectura general	Sí	Sí	No
Matrículas: alta y anulación	Sí	No	No
GET /api/matriculas/mias	No	No	Sí
Calificaciones: lectura y escritura	Sí	Sí	No
GET /api/calificaciones/mias	No	No	Sí

Evidencias de seguridad:

Petición	Resultado	Código que lo explica
POST /auth/login (admin)	200 con token	AuthServiceImpl.login, JwtUtils.generateToken
POST /auth/login con clave errónea	401 AUTH_INVALID_CREDENTIALS	InvalidCredentialsException
GET /api/estudiantes sin header	401 AUTH_HEADER_MISSING	AuthValidationFilter, CustomAuthenticationEntryPoint
GET /api/estudiantes con token alterado	401 TOKEN_INVALID	AuthValidationFilter (FeignException.Unauthorized)
POST /api/cursos con token docente	403 AUTH_FORBIDDEN	SecurityConfig, CustomAccessDeniedHandler



Swagger en el puerto 8081: POST /auth/login con admin, respuesta 200 con el token

Curl

```
curl -X 'POST' \
  'http://localhost:8081/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "estudiante",
  "password": "estudiante3"
}'
```

Request URL

```
http://localhost:8081/auth/login
```

Server response

Code	Details
401 *Undocumented*	Error: response status is 401

Response body

```
{
  "success": false,
  "message": "Credenciales inválidas",
  "errorCode": "AUTH_INVALID_CREDENTIALS",
  "status": 401,
  "path": "/auth/login",
  "timestamp": "2026-11-06T21:00:01.659855203"
}
```

Download

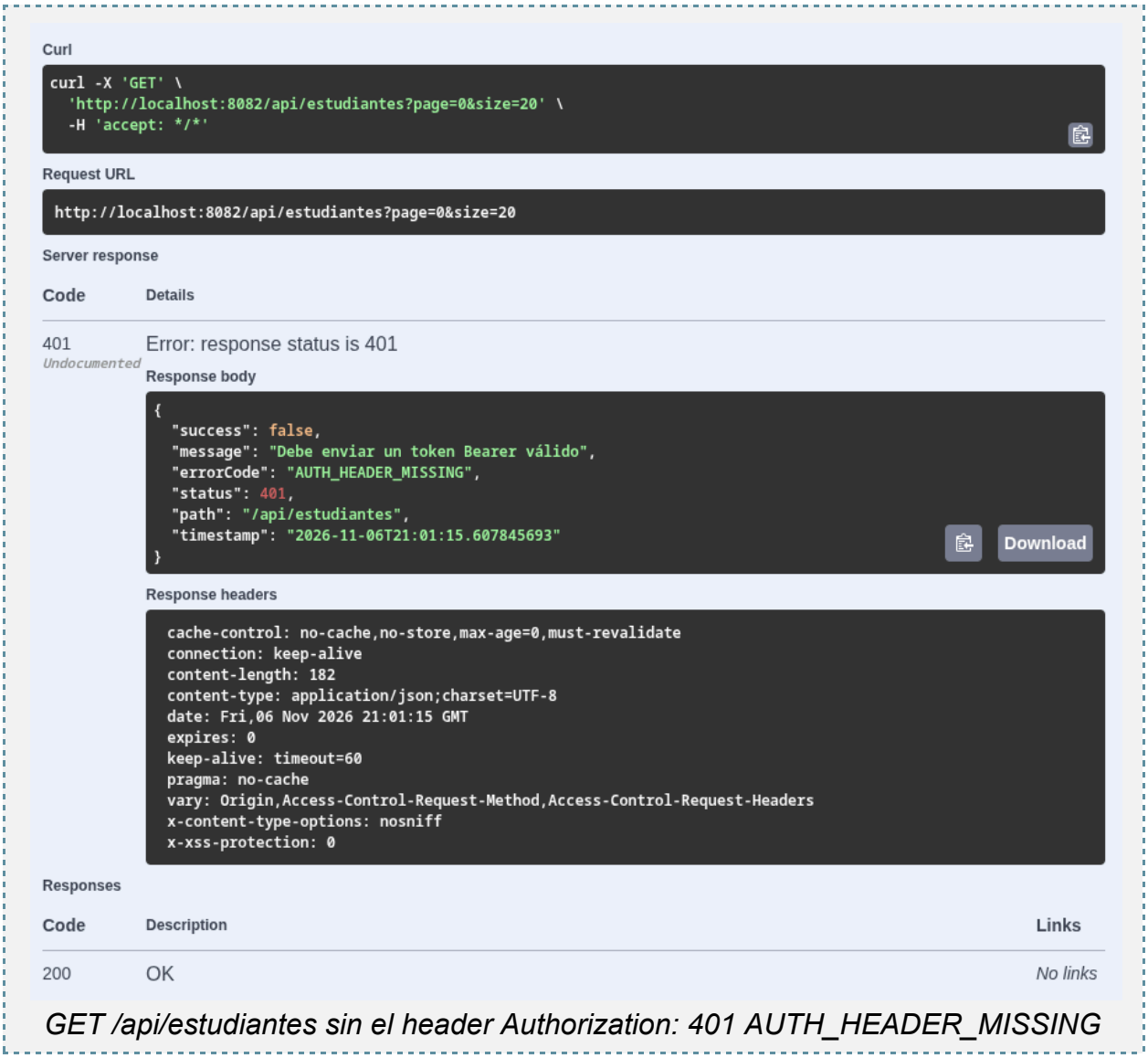
Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Fri,06 Nov 2026 21:00:01 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK	*No links*

POST /auth/login con contraseña incorrecta: 401 AUTH_INVALID_CREDENTIALS



Curl

```
curl -X 'GET' \
  'http://localhost:8082/api/estudiantes?page=0&size=20' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8082/api/estudiantes?page=0&size=20
```

Server response

Code	Details
401 *Undocumented*	Error: response status is 401

Response body

```
{
  "success": false,
  "message": "Debe enviar un token Bearer válido",
  "errorCode": "AUTH_HEADER_MISSING",
  "status": 401,
  "path": "/api/estudiantes",
  "timestamp": "2026-11-06T21:01:15.607845693"
}
```

Download

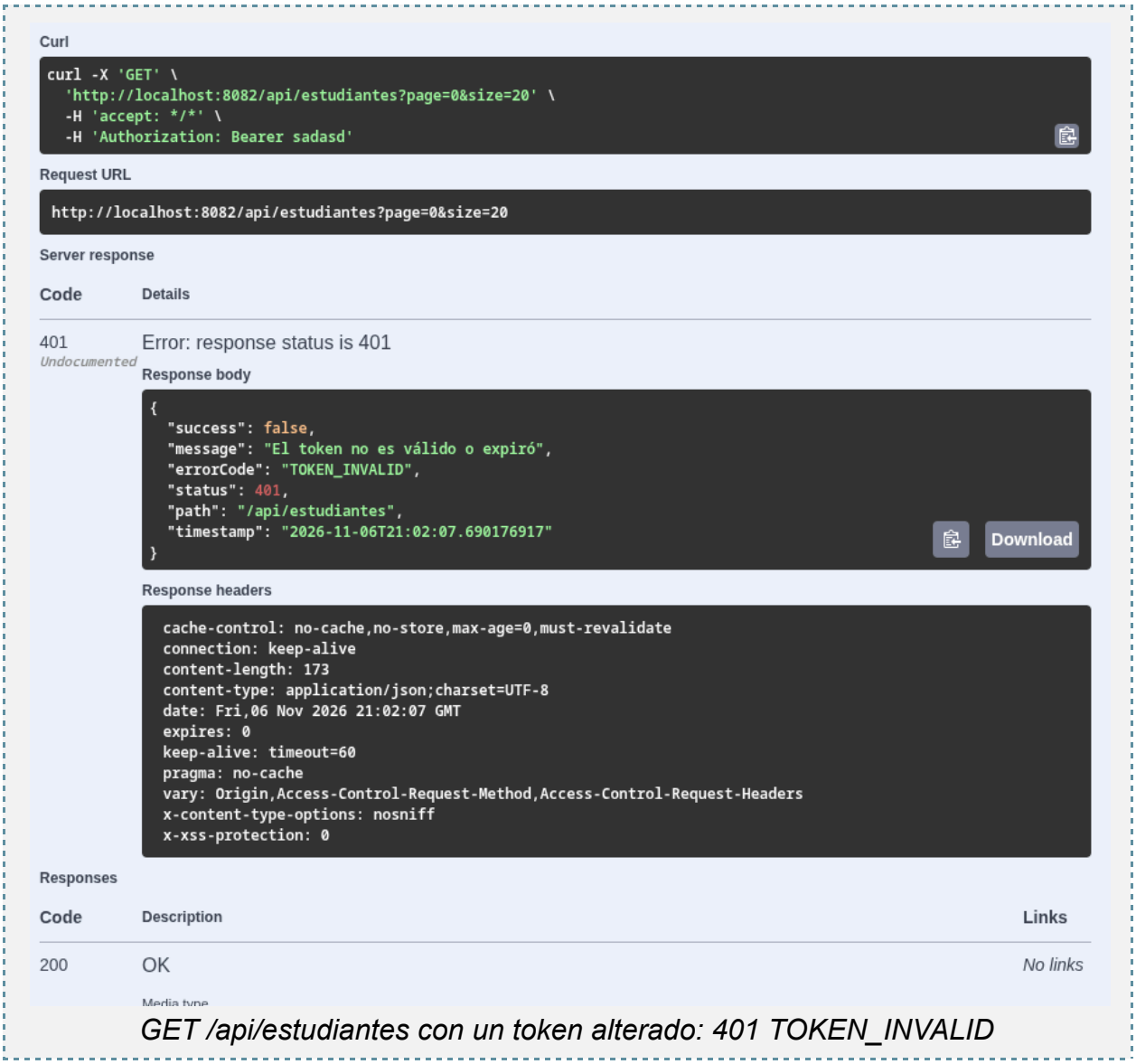
Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 182
content-type: application/json;charset=UTF-8
date: Fri,06 Nov 2026 21:01:15 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-xss-protection: 0
```

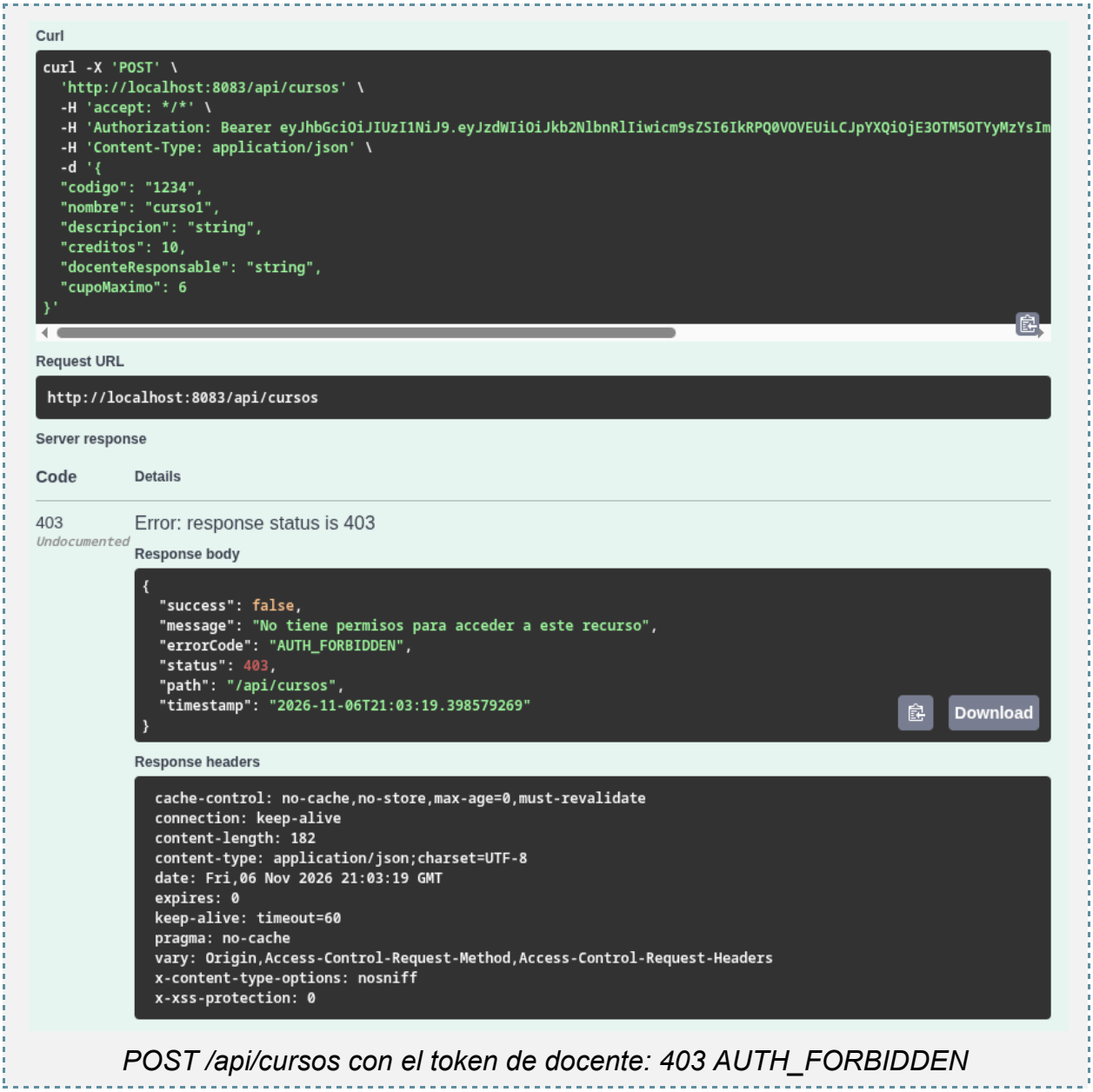
Responses

Code	Description	Links
200	OK	*No links*

GET /api/estudiantes sin el header Authorization: 401 AUTH_HEADER_MISSING



GET /api/estudiantes con un token alterado: 401 TOKEN_INVALID



POST /api/cursos con el token de docente: 403 AUTH_FORBIDDEN

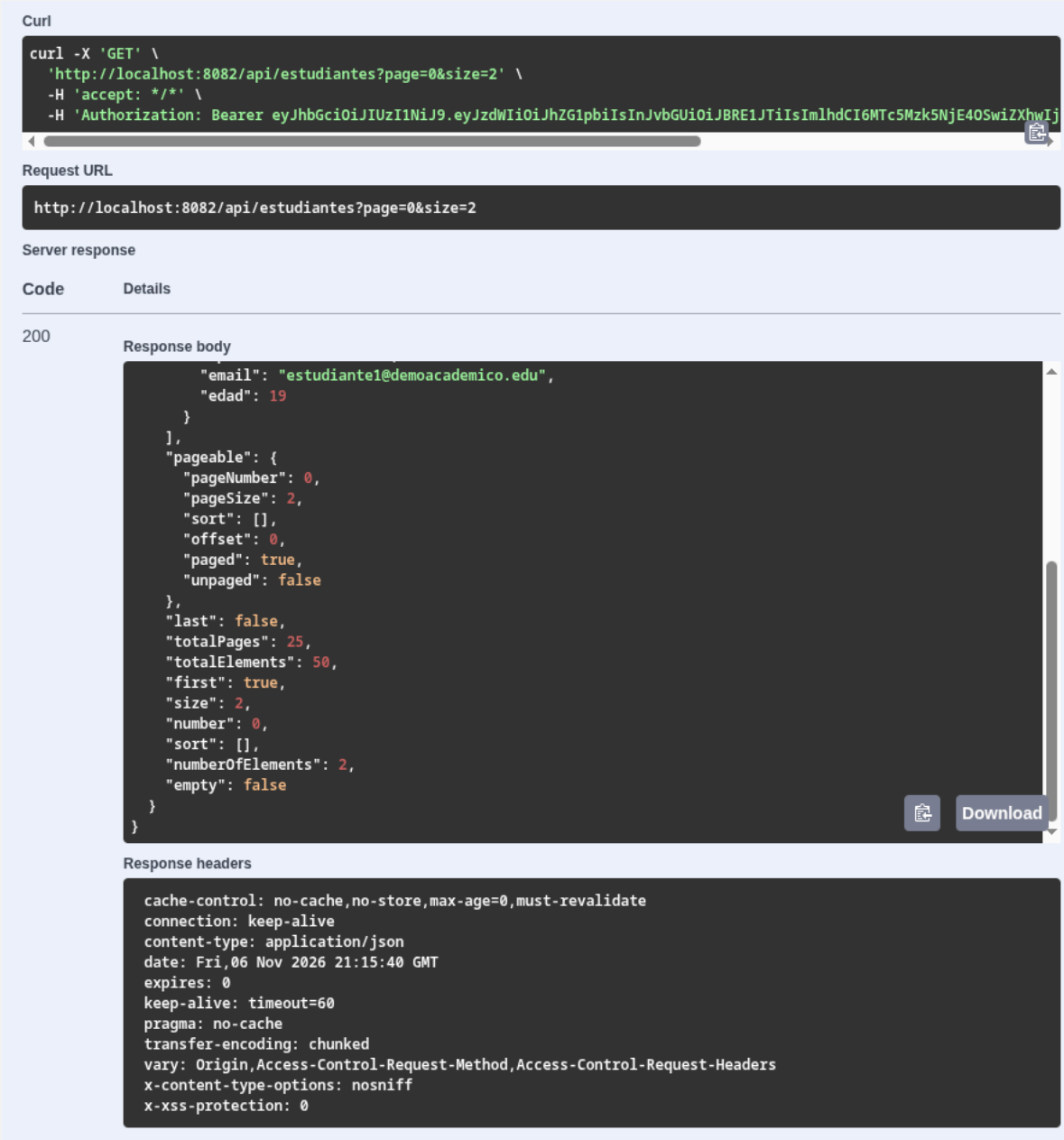
5. Módulo de estudiantes

POST /api/estudiantes recorre AuthValidationFilter, EstudianteController (con @Valid), EstudianteHandler, EstudianteServiceImpl.crear y EstudianteRepository. El servicio verifica el correo con existsByEmail y responde 400 si ya existe; la restricción única uk_estudiante_email cubre el caso de dos altas simultáneas, que se traduce en un 409 mediante DataIntegrityViolationException.

El DTO de entrada no tiene id, por lo que el cliente no asigna la identidad, y la validación (@NotBlank, @Email, @Min) ocurre en el borde HTTP. El listado recibe un

Pageable y devuelve una página; sin paginación, un catálogo grande se cargaría completo en memoria.

Petición	Resultado	Código que lo explica
GET /api/estudiantes?page=0&size=2	200, totalElements 50	EstudianteController, Pageable, EstudianteRepository



GET /api/estudiantes?page=0&size=2: respuesta con totalElements 50

6. Módulo de cursos

Cursos replica el corte de capas de estudiantes. Aplica código único, créditos entre 1 y 10 y cupo de entrada entre 1 y 60. La lectura del catálogo está abierta al rol ESTUDIANTE, que lo necesita para elegir curso; el directorio de estudiantes, en cambio, solo se abre a ADMIN y DOCENTE porque expone datos de terceros.

El borrado de un curso y la reducción de cupoMaximo por debajo de las matrículas activas dependen de matrículas-service. Si este no responde, esas dos rutas lanzan RemoteServiceException y devuelven 502; crear, listar y aumentar el cupo siguen funcionando sobre la base local.

Petición	Resultado	Código que lo explica
GET /api/cursos con token estudiante	200	SecurityConfig de cursos
GET /api/estudiantes con token estudiante	403	SecurityConfig de estudiantes

Curl

```
curl -X 'GET' \
  'http://localhost:8083/api/cursos?page=0&size=20' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc3R1ZGlhbnRlIiwicm9sZSI6IkVTVFVESUFOVEUiLCJpYXQiOjE3OTM5OT
```

Request URL

```
http://localhost:8083/api/cursos?page=0&size=20
```

Server response

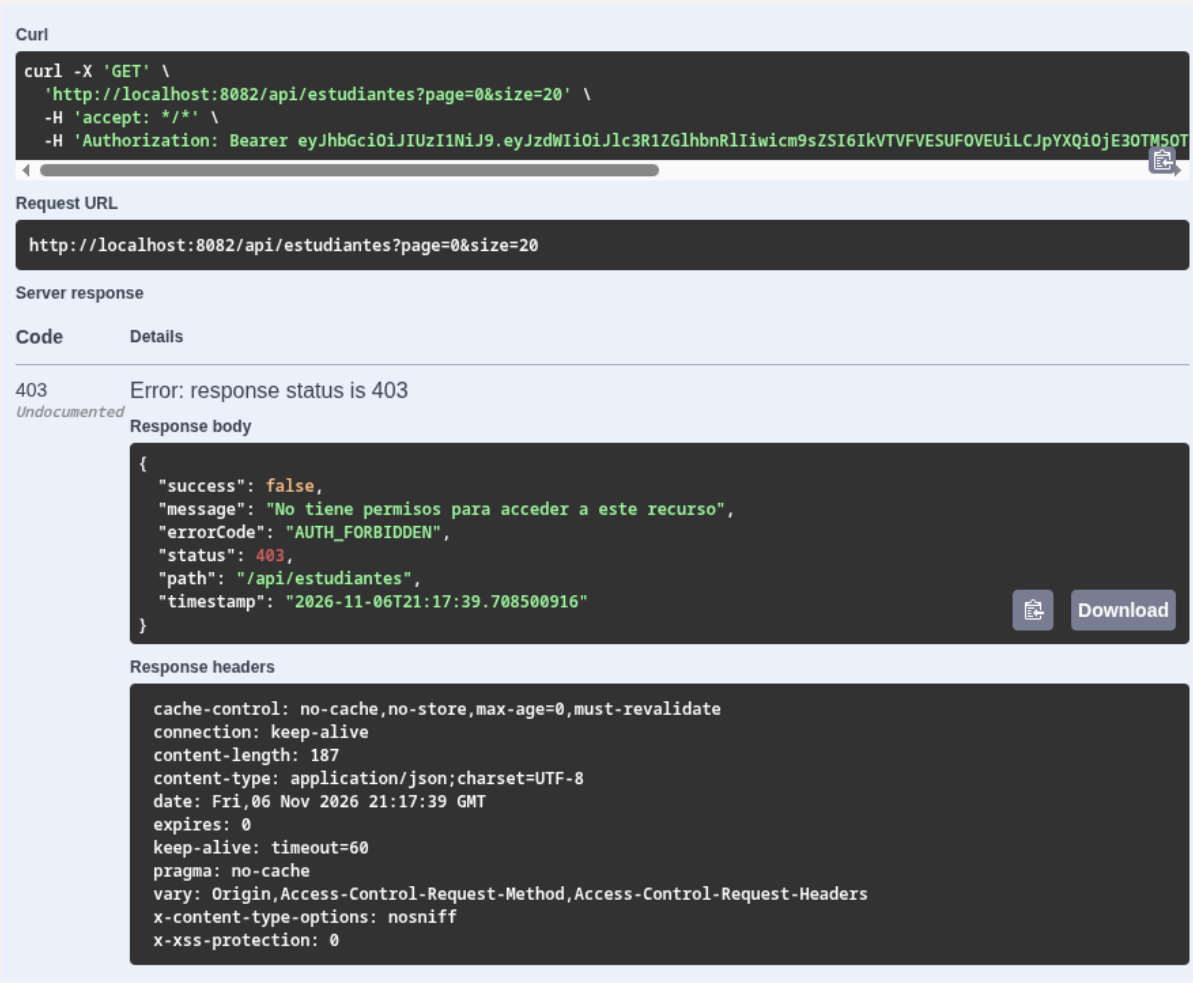
Code	Details
200	Response body


```
{
  "success": true,
  "message": "OK",
  "data": {
    "content": [
      {
        "id": 1,
        "codigo": "IS3-001",
        "nombre": "Ingeniería de Software III",
        "descripcion": "Arquitectura y microservicios",
        "creditos": 4,
        "docenteResponsable": "Docente principal",
        "cupoMaximo": 30
      },
      {
        "id": 2,
        "codigo": "BD2-001",
        "nombre": "Bases de Datos II",
        "descripcion": "Modelado avanzado y SQL",
        "creditos": 3,
        "docenteResponsable": "Docente de bases de datos",
        "cupoMaximo": 25
      },
```


Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Fri,06 Nov 2026 21:16:50 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-xss-protection: 0
```



Con token de estudiante: GET /api/cursos responde 200 y GET /api/estudiantes responde 403

7. Módulo de matrículas y comunicación entre microservicios

La entidad Matricula guarda estudianteId y cursoId como Long, porque las filas viven en otras bases, y el estado como enumeración almacenada en texto (ACTIVA o ANULADA). POST /api/matriculas ejecuta, en orden: lectura del estudiante por Feign, lectura del curso por Feign (de donde sale cupoMaximo), verificación de duplicado activo, conteo de matrículas activas y alta con estado ACTIVA.

El alta implica seis llamadas HTTP síncronas en serie: la del cliente, tres validaciones en auth-service (una por cada servicio que recibe el Bearer) y las lecturas a estudiantes y a cursos. FeignAuthPropagationConfig copia el header Authorization desde RequestContextHolder hacia las llamadas Feign. Sin esa copia, estudiantes

respondería 401 y matrículas lo convertiría en un 502 aunque el usuario hubiera enviado un token válido.

Traducción de errores remotos: un 404 de estudiantes o cursos se convierte en BusinessException (400, el cliente debe corregir el identificador); cualquier otra falla Feign, incluida la conexión rechazada (código -1), se convierte en RemoteServiceException (502). La anulación (PUT /{id}/anular) cambia el estado a ANULADA y conserva la fila; no existe DELETE.

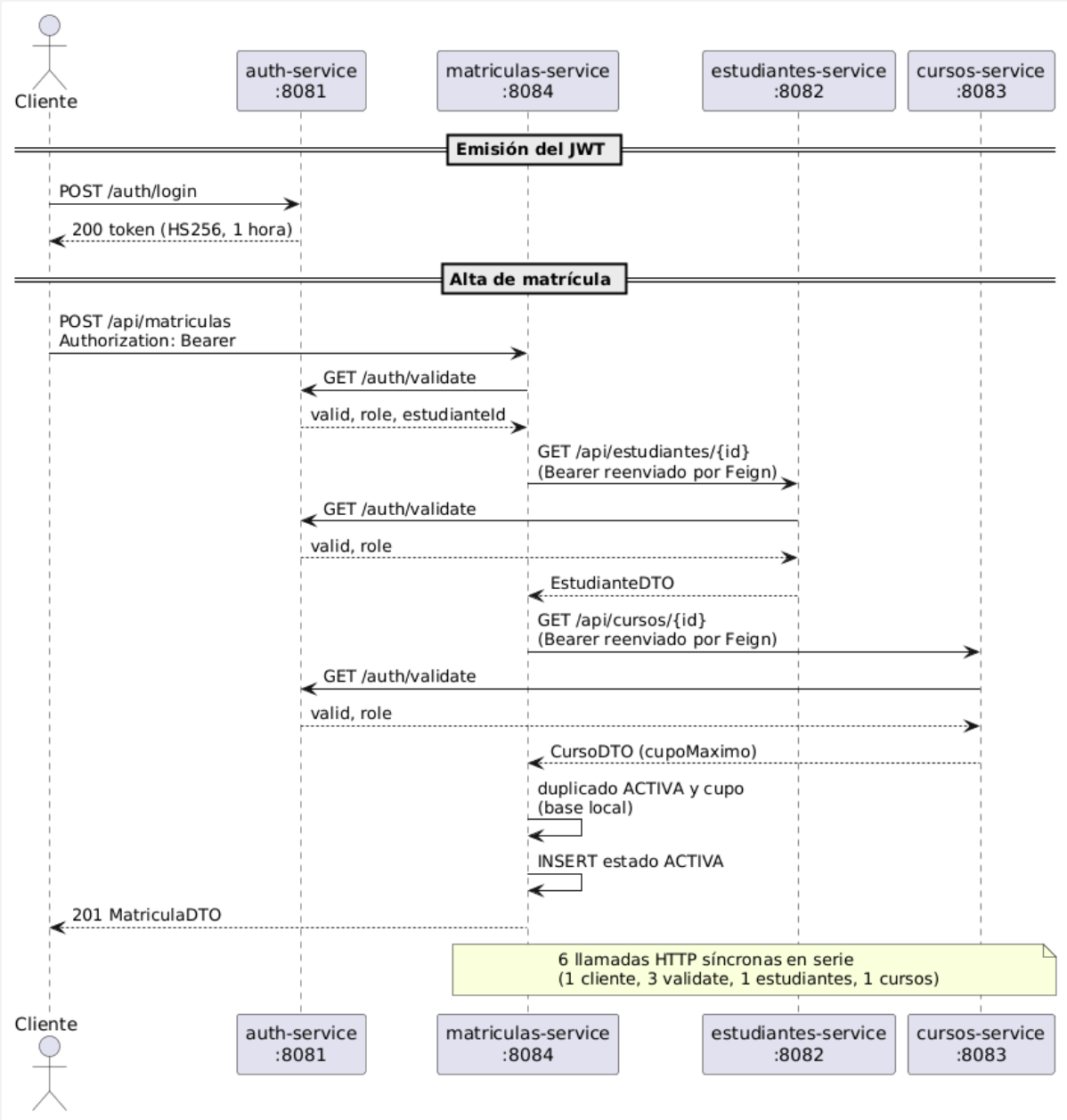
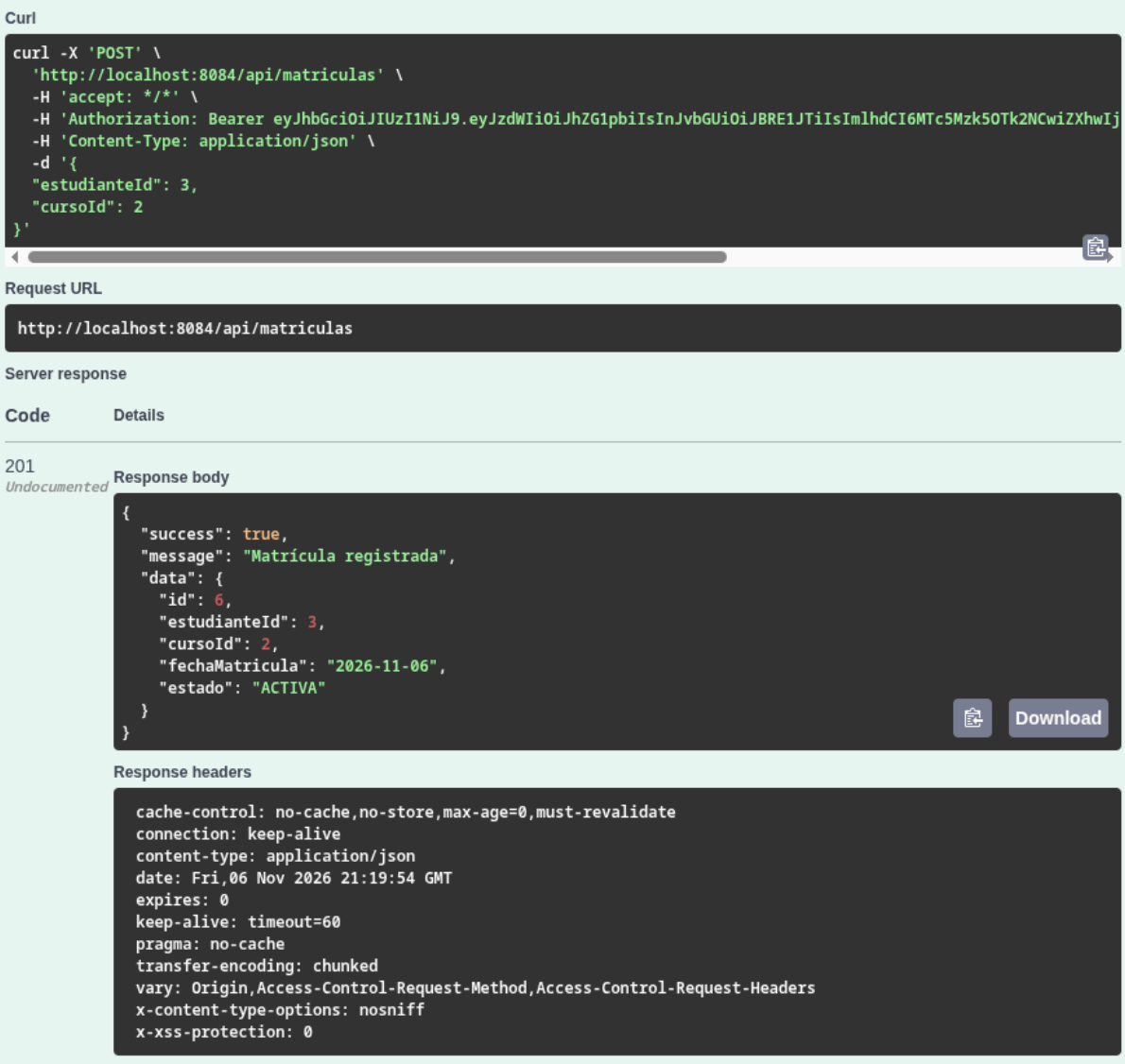


Diagrama de secuencia: login, emisión del JWT y las 6 llamadas HTTP del POST /api/matriculas

Petición	Resultado	Código que lo explica
POST /api/matriculas (admin)	201, estado ACTIVA	MatriculaServiceImpl.registrar
POST /api/matriculas con cursos-service detenido	502, código remoto -1	MatriculaServiceImpl.obtenerRemoto, RemoteServiceException
PUT /api/matriculas/3/anular dos veces	Primera: ANULADA; segunda: 400	MatriculaServiceImpl.anular



POST /api/matriculas con token admin: 201 y estado ACTIVA

Curl

```
curl -X 'POST' \
  'http://localhost:8084/api/matriculas' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsInJvbGUiOiJBRE1JTiIsImlhdCI6MTc5Mzk5OTk2NCwiZXhwIj
  -H 'Content-Type: application/json' \
  -d '{
  "estudianteId": 3,
  "cursoId": 2
}'
```

Request URL

```
http://localhost:8084/api/matriculas
```

Server response

Code	Details
502 *Undocumented*	Error: response status is 502

Response body

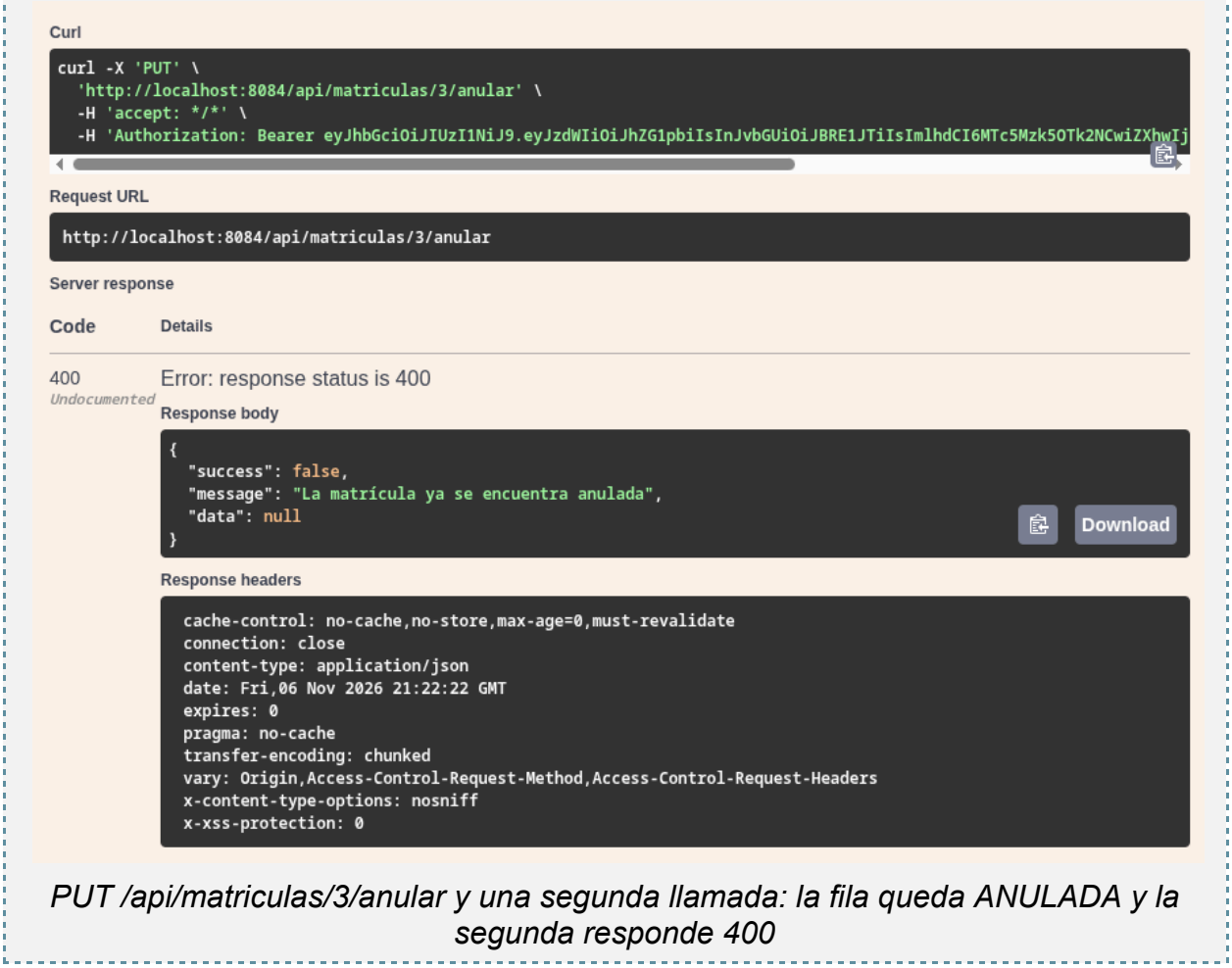
```
{
  "success": false,
  "message": "No fue posible consultar el curso (código remoto -1)",
  "data": null
}
```

Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Fri,06 Nov 2026 21:20:58 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-xss-protection: 0
```

Con cursos-service detenido (docker compose stop cursos-service), repetir el alta: 502 con «código remoto -1»



PUT /api/matriculas/3/anular y una segunda llamada: la fila queda ANULADA y la segunda responde 400

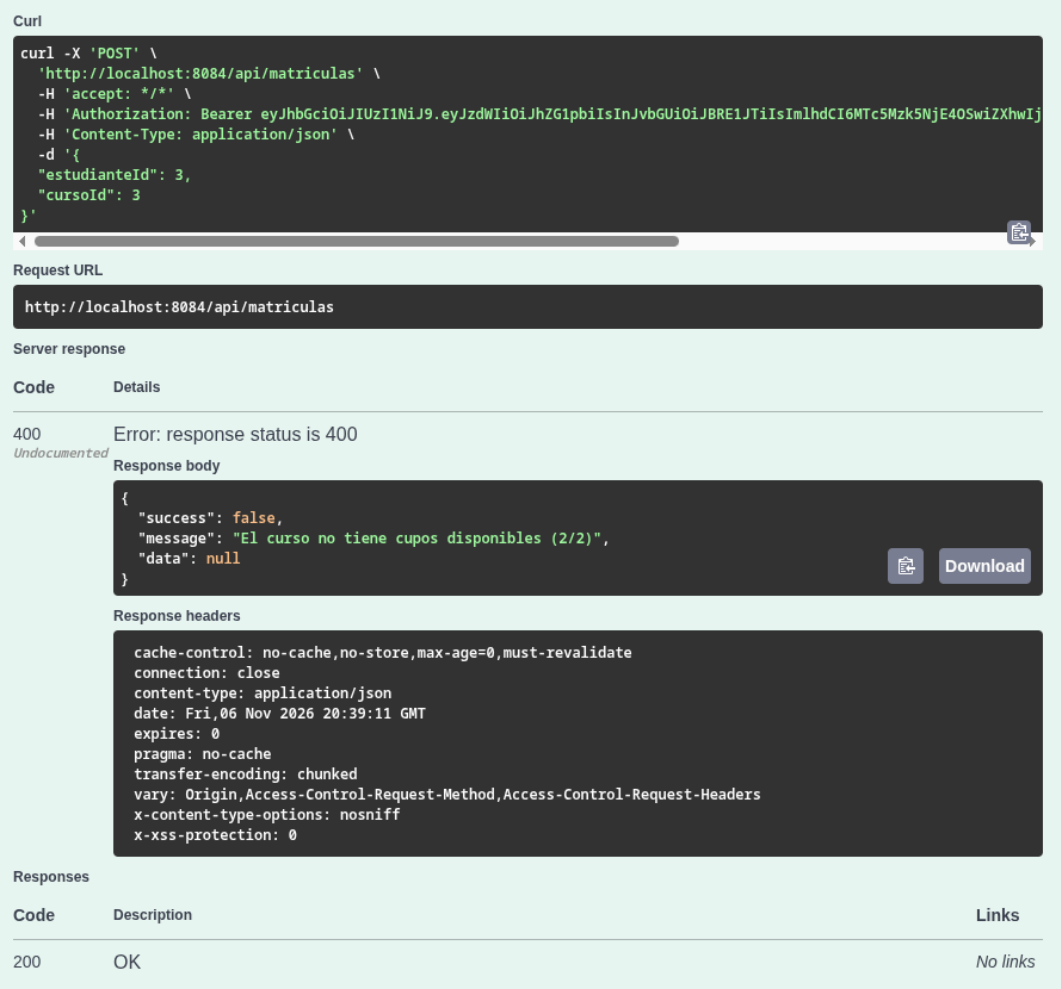
8. Extensión E2: control de cupos

El cupo máximo es un atributo de la oferta y lo posee cursos; el número de matrículas activas es un hecho de la matrícula y lo posee matrículas. Guardar una copia del cupo en matrículas la dejaría desactualizada cuando cursos lo reduzca. Por eso registrar consulta el cupo por Feign y cuenta las activas con countByCursoIdAndEstado.

El duplicado se evalúa antes que el cupo, para que un estudiante ya matriculado reciba el mensaje de matrícula existente y no el de curso lleno. CursoServiceImpl.actualizar consulta matrículas solo cuando el cupo nuevo es menor que el actual.

Limitación analizada: registrar lee el conteo y luego inserta, sin bloqueo ni restricción sobre la cantidad de filas ACTIVA por curso. En teoría, dos altas simultáneas del último cupo podrían confirmarse las dos. Este escenario se deduce del código y no se reprodujo en la verificación.

Petición	Resultado	Código que lo explica
Tercera matrícula en RED-001 (cupo 2)	400, mensaje con 2/2	MatriculaServiceImpl.registrar
PUT del cupo de RED-001 a 1 con 2 activas	400 en cursos	CursoServiceImpl.actualizar



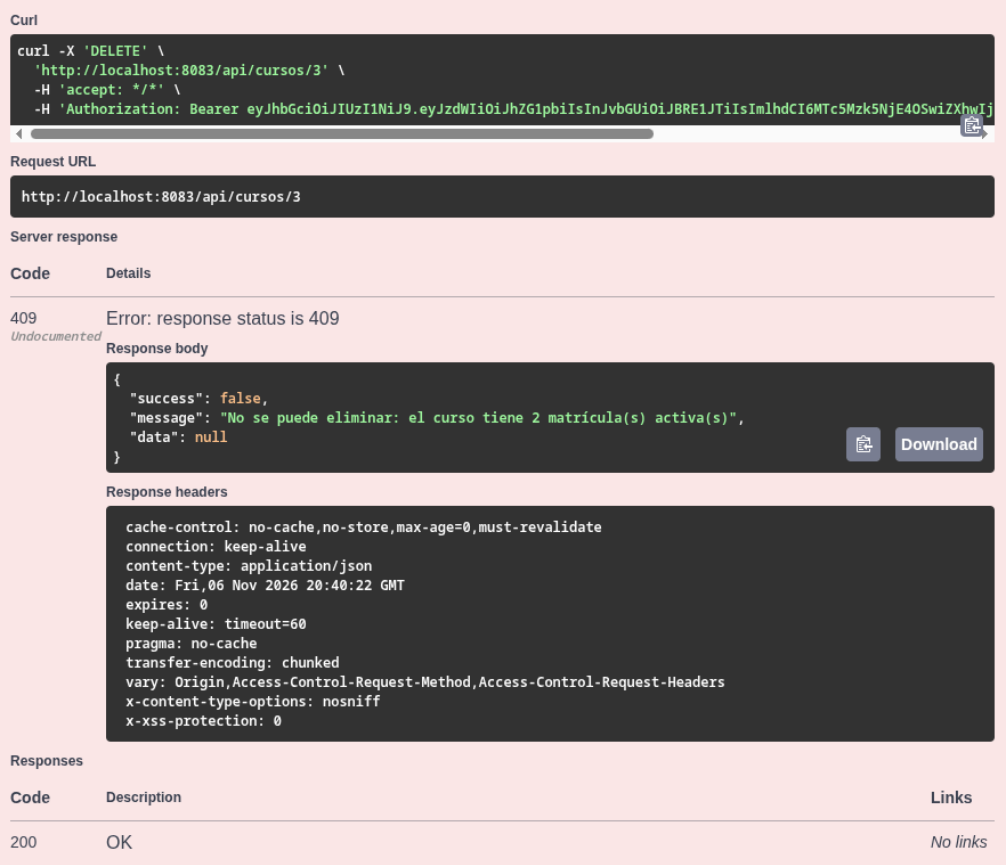
POST /api/matriculas en RED-001 con dos matrículas activas: 400 con «2/2»

9. Extensión E1: integridad entre servicios

Estudiantes y cursos consultan GET /api/matriculas/activas/conteo antes de borrar, y cursos también al reducir el cupo. Calificaciones consulta matrículas en el alta, la corrección, el resumen y /mias. Si matrículas no responde, el borrado lanza RemoteServiceException antes del delete: es una decisión fail-closed, pues borrar sin saber si hay alumnos inscritos sería peor que no completar el borrado.

El borrado de un recurso con matrículas activas devuelve 409 (ConflictException): el recurso existe y su estado impide la operación, y el cliente puede anular las matrículas y reintentar. El 400 se reserva para cuerpos o reglas de entrada inválidos. contarActivas exige exactamente uno de estudianteId o cursoId; ninguno o los dos producen 400.

Petición	Resultado	Código que lo explica
DELETE del curso RED-001 con matrículas activas	409	CursoServiceImpl.eliminar, ConflictException
DELETE del estudiante 1 con matrícula activa	409	EstudianteServiceImpl.eliminar



DELETE /api/cursos/{id de RED-001} con matrículas activas: 409

10. Extensión E4: mis matrículas

Un GET /api/matriculas?estudianteId=1 abierto al rol ESTUDIANTE sería un IDOR, pues cambiar el número permitiría leer matrículas ajenas. GET /api/matriculas/mias no

recibe ese parámetro: el controlador toma @AuthenticationPrincipal UsuarioAutenticado y usa usuario.estudianteId(), que proviene del claim del token.

En SecurityConfig la regla de /mias se declara antes del comodín /api/matriculas/**, porque Spring Security aplica la primera regla que coincide. UsuarioAutenticado es un record que implementa Principal, de modo que getName() devuelve el username y el id queda tipado. El estudianteId vive hasta la expiración del token, así que un cambio de asociación solo se refleja en el siguiente login.

Petición	Resultado	Código que lo explica
GET /mias?estudianteId=99 con token estudiante	200, solo la matrícula del estudiante 1	MatriculaController.misMatriculas
GET /mias con token ADMIN	403 AUTH_FORBIDDEN	SecurityConfig de matrículas



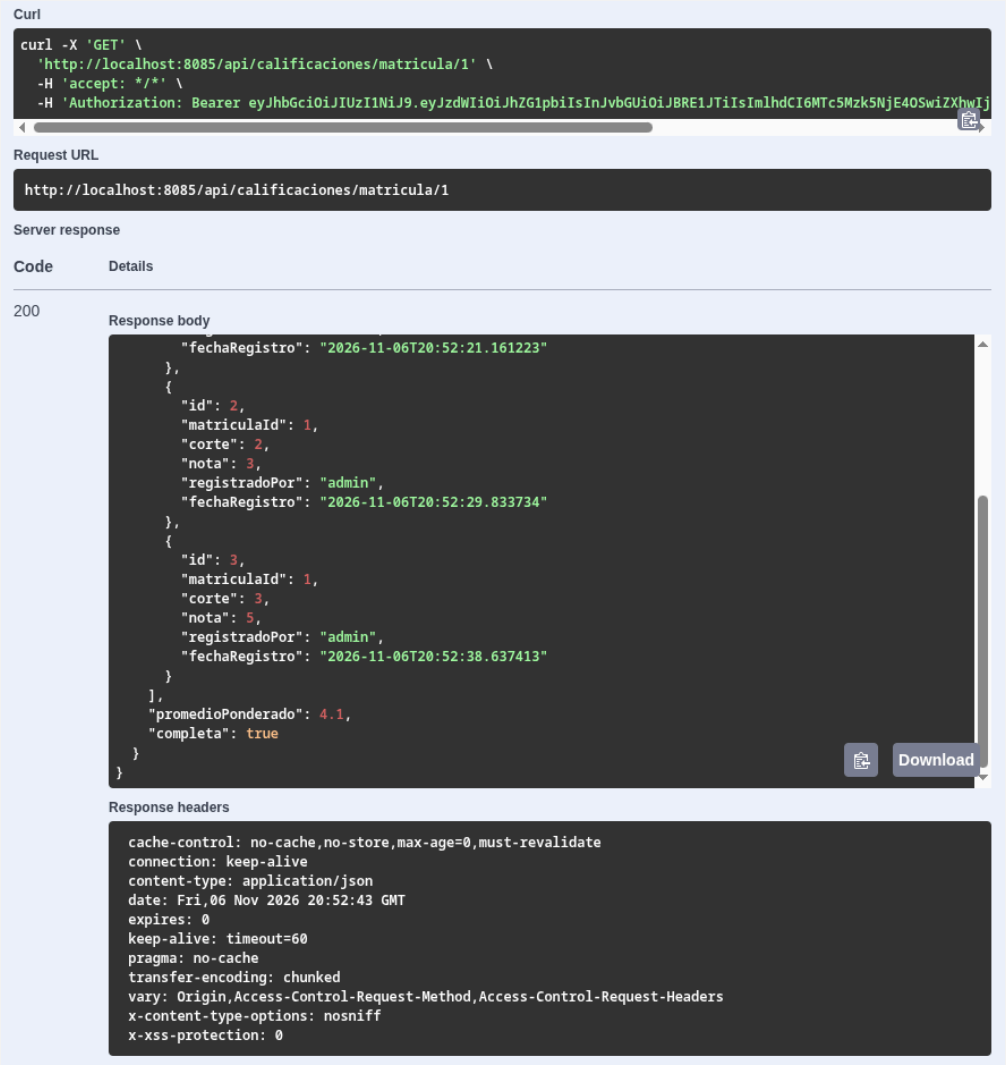
GET /api/matriculas/mias?estudianteId=99 con el token de estudiante: solo aparece la matrícula del estudiante 1

11. Extensión E3: calificaciones

Calificacion guarda matriculaId como Long, el corte (1, 2 o 3) y la nota como BigDecimal con un decimal. La restricción uk_calificacion_matricula_corte impide dos notas para el mismo corte. Los pesos son 0.30, 0.30 y 0.40: calcularPromedio suma nota por peso de los cortes presentes y divide por la suma de esos pesos, con escala 2 y HALF_UP. El promedio no se almacena y completa() es verdadero solo con los tres cortes.

El autor de la nota no viaja en el DTO: el controlador lo toma de authentication.getName(). Anular una matrícula no borra sus notas, pero impide registrar o corregir nuevas. Un estudiante solo puede consultar sus notas por /api/calificaciones/mias, que llama a /api/matriculas/mias con el Bearer propagado.

Petición	Resultado	Código que lo explica
Notas 4.0, 3.0 y 5.0 en la matrícula 1	Promedio 4.10, completa=true	CalificacionServiceImpl.calcularPromedio
PUT del corte 1 a 4.5	Promedio 4.25	CalificacionServiceImpl
POST sobre la matrícula 3 (anulada)	400	CalificacionServiceImpl.registrar
Token estudiante en GET /matricula/1	403	SecurityConfig de calificaciones



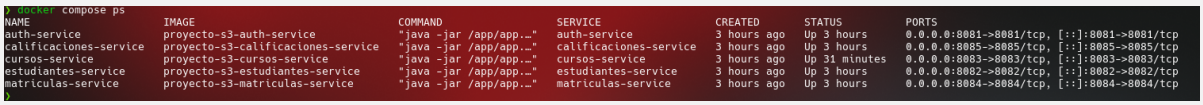
GET /api/calificaciones/matricula/1 con las tres notas: promedio 4.10 y completa=true

12. Docker y pruebas

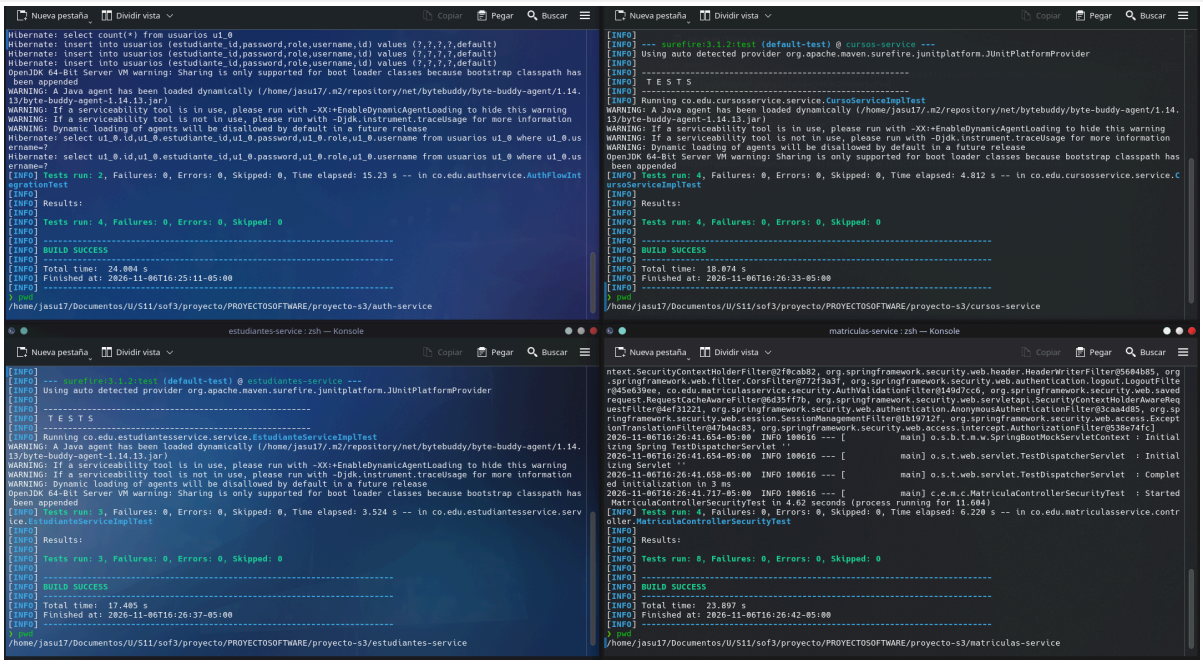
Dentro de la red proyecto-s3_default, las URLs usan el nombre DNS del servicio (por ejemplo http://auth-service:8081); localhost dentro de un contenedor es el propio contenedor. depends_on ordena la creación, pero no espera a que el puerto acepte tráfico porque no hay healthcheck. El Dockerfile copia un JAR ya empaquetado en un contenedor Maven con JDK 21, de modo que el host no necesita Maven.

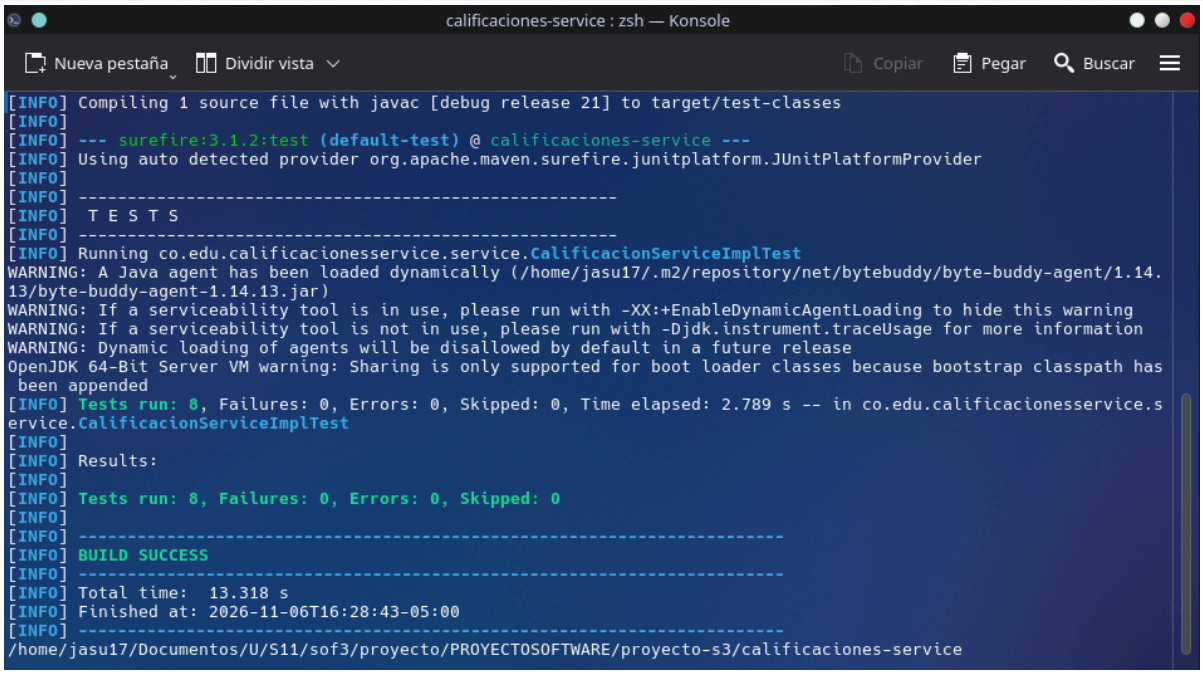
Al ser H2 en memoria, reiniciar un contenedor vacía su base y la semilla se carga de nuevo si la tabla está vacía. Las 27 pruebas automáticas sustituyen las llamadas Feign por mocks, así que no detectan problemas de red entre contenedores ni la concurrencia del cupo; esos casos se verificaron con el stack en ejecución.

Módulo	Casos	Clases de prueba	Resultado
auth-service	4	JwtUtilsTest, AuthFlowIntegrationTest	BUILD SUCCESS
estudiantes-service	3	EstudianteServiceImplTest	BUILD SUCCESS
cursos-service	4	CursoServiceImplTest	BUILD SUCCESS
matriculas-service	8	MatriculaServiceImplTest, MatriculaControllerSecurityTest	BUILD SUCCESS
calificaciones-service	8	CalificacionServiceImplTest	BUILD SUCCESS
Total	27		0 fallos, 0 errores



docker compose ps: los cinco contenedores en estado Up y los puertos 8081 a 8085





mvn test en los cinco módulos: BUILD SUCCESS y el total de 27 casos

13. Gestión de datos y calidad del código

Cada servicio usa jdbc:h2:mem con ddl-auto en update y no hay claves foráneas entre procesos. Las restricciones únicas son uk_estudiante_email (Estudiante.email),

uk_curso_codigo (Curso.codigo) y uk_calificacion_matricula_corte (Calificacion.matricula_id y corte). El aislamiento transaccional es el de cada base local: entre servicios, la lectura HTTP y la escritura posterior no comparten transacción.

En calidad del código, cada servicio es cohesivo en torno a un agregado y el acoplamiento entre procesos es el contrato JSON, no clases compartidas. Como costo, AuthValidationFilter, GlobalExceptionHandler y ResponseBuilder están copiados en los servicios, lo que puede producir deriva entre copias, y EstudianteCreateDTO y EstudianteUpdateDTO tienen hoy los mismos campos. Ninguna clase supera las 150 líneas; la más larga es CalificacionServiceImpl con 139.

14. Oportunidades de mejora

Mejora	Problema observado	Propuesta	Beneficio
API Gateway	Se publican cinco puertos y cada servicio repite la comprobación del Bearer.	Spring Cloud Gateway como único puerto, con rutas por path.	Un punto para TLS y límite de tasa; el cliente deja de conocer los puertos.
Resiliencia	Si un destino cae, Feign falla de inmediato con 502 y sin reintentos.	Resilience4j: circuit breaker y retry en los clientes Feign.	Menos hilos bloqueados y distinción entre circuito abierto y error de dominio.
Mensajería de dominio	El alta encadena seis HTTP síncronos y bloquea el hilo.	Un broker y un evento MatriculaRegistrada para los efectos posteriores.	Menor latencia en el camino de respuesta; consumidores escalables.
JWT RS256	Cada petición llama a /auth/validate y el secreto HS256 es un literal.	Firma asimétrica: validar en local con la clave pública.	Se eliminan 3 de las 6 llamadas del alta y auth deja de ser un punto único de falla.
Cupo atómico	El cupo se decide con lectura y escritura sin bloqueo.	Bloqueo por curso o inserción condicionada en registrar.	Dos altas simultáneas dejan una sola fila ACTIVA.

15. Conclusiones

- La separación en cinco servicios con base propia permite desplegar cada agregado por separado, a cambio de consistencia diferida entre servicios.
- La seguridad delega la firma en auth-service y aplica los roles en cada servicio; el diseño es claro, pero el secreto en texto plano y la llamada de validación por petición son sus puntos débiles.
- Las reglas de negocio verificadas (cupo, integridad, mis matrículas y promedio) responden como se espera, y las pruebas automáticas dejan 27 casos sin fallos.
- Las mejoras propuestas atacan la disponibilidad, el rendimiento, la seguridad y la consistencia del sistema.

Repositorio_Proyecto

Anexo. Lista de capturas y diagramas

Elemento	Contenido	Sección
Captura 1	POST /auth/login con admin: 200 y token	4
Captura 2	Login con clave errónea: 401 AUTH_INVALID_CREDENTIALS	4
Captura 3	Sin header Authorization: 401 AUTH_HEADER_MISSING	4
Captura 4	Token alterado: 401 TOKEN_INVALID	4
Captura 5	POST /api/cursos con docente: 403	4
Captura 6	Listado paginado: totalElements 50	5
Captura 7	Token estudiante: cursos 200 y estudiantes 403	6
Captura 8	Alta de matrícula: 201 ACTIVA	7
Captura 9	cursos-service detenido: 502 código remoto -1	7
Captura 10	Anulación y segunda anulación: 400	7
Captura 11	Cupo de RED-001: 400 con 2/2	8
Captura 12	Borrado con matrículas activas: 409	9
Captura 13	/mias con ?estudianteId=99: solo el estudiante 1	10
Captura 14	Promedio 4.10 y completa=true	11
Captura 15	docker compose ps	12

Elemento	Contenido	Sección
Captura 16	mvn test en los cinco módulos	12
Diagrama 1	Componentes y flujo del token (PlantUML 1)	3
Diagrama 2	Secuencia del alta de matrícula (PlantUML 2)	7